

Számítási és tárolási felhők alapjai (VIIIMA26)
Gyakorlat

Ludmány Balázs

2025/26 őszi

Tartalomjegyzék

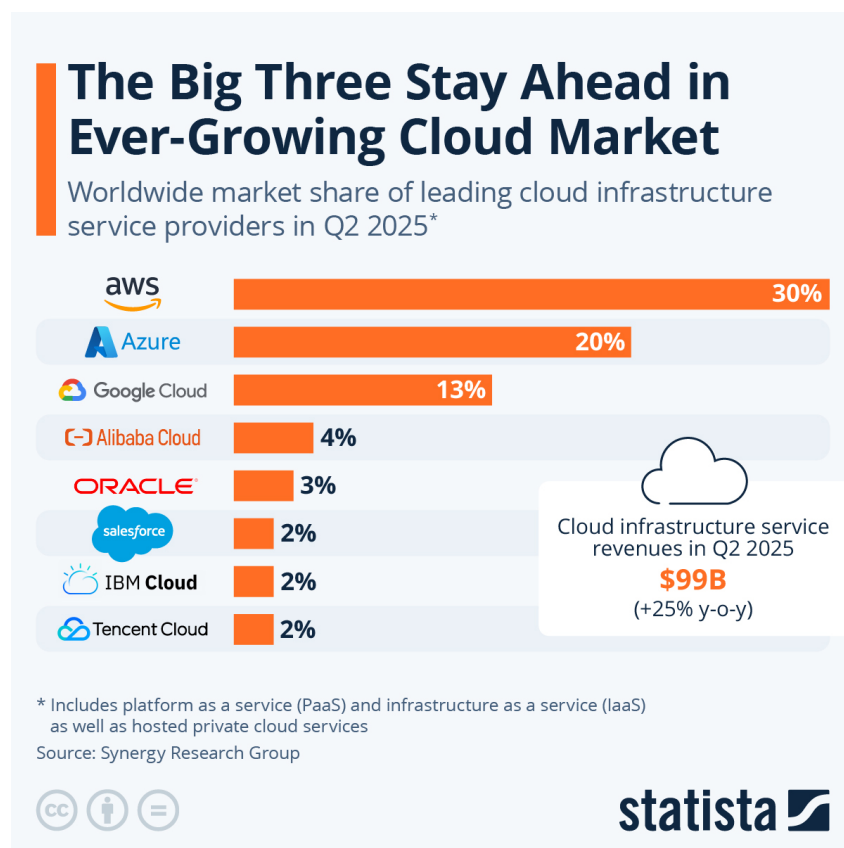
Bevezetés	5
1. Miért pont AWS?	5
1. Szolgáltatási rétegek	7
1. Amazon Resource Name (ARN)	7
2. Első lépések	8
2.1. Feladat: regisztráció és bejelentkezés az AWS-be	8
2.2. AWS Management Console	9
2.3. Feladat: kiadások követése a Management Console-on	10
3. AWS Identity and Access Management (IAM)	10
3.1. Alapfogalmak	10
3.2. Műveletek az AWS-ben	11
4. Infrastruktúra-szolgáltatás: Elastic Compute Cloud (EC2)	13
4.1. A virtuális gép személyre szabása	13
5. Szoftverszolgáltatás: Rocket.Chat	15
5.1. Mikroszolgáltatás architektúra	15
6. Platformszolgáltatás: Elastic Kubernetes Service	16
6.1. Alkalmazáskonténerek	16
6.2. Konténer orkesztráció	16
6.3. EKS Auto Mode	17
6.4. Parancssori eszközök: <code>aws</code> , <code>eksctl</code> , <code>kubectl</code> és Helm	18
7. Feladatok	18
7.1. IAM	18
7.2. Átjelentkezés a <code>developer</code> fiókba	19
7.3. SSH hozzáférés	19
7.4. Virtuális gép indítása	21
7.5. Bejelentkezés a PuTTY SSH klienssel	22
7.6. Kubernetes klaszter létrehozása Auto Mode-ban	23
7.7. A klaszter konfigurálása és a Rocket.Chat telepítése	23
7.8. A Rocket.Chat indítása	24
7.9. A klaszter eltávolítása	24
2. Automatikus skálázás	27
1. Klaszter konfigurálása részletesebben	27
1.1. Feladat: a Kubernetes klaszter újraindítása	27
1.2. Deklaratív konfiguráció és YAML	29
1.3. Feladat: Rocket.Chat telepítése (újra)	29
2. Terheléspróba	30
2.1. Rocket.Chat REST API	30

2.2.	Locust	30
2.3.	Feladat	32
3.	Állapotmentes mikroszolgáltatások automatikus skálázása	33
3.1.	Alapfogalmak	33
3.2.	A korlátlan automatikus skálázódás veszélyei	34
3.3.	Feladat: manuális skálázás	35
3.4.	Feladat: automatikus skálázás	35
3.	Infrastructure-as-Code	37
1.	Feladat	37

Bevezetés

1. Miért pont AWS?

A felhőszolgáltatások piacát három nagy szolgáltató uralja, az Amazon Web Services (AWS), a Microsoft Azure és a Google Cloud Platform (GCP). Közülük is a legnagyobb az AWS, a teljes piac kb. 30%-át teszi ki. Ezért esett a választás az AWS-re, hiszen statisztikailag a későbbiekben ezzel van legnagyobb esélyetek találkozni. Fontos tehát kiemelni, hogy semmilyen minőségbeli szempont nem áll a választásunk mögött, nem állítjuk, hogy az AWS bármelyik másik szolgáltatónál jobb vagy rosszabb lenne.



1. ábra. Felhőszolgáltatók piaci részesedése 2025 második negyedévében. Forrás

A piaci felhőszolgáltatásokon kívül érdemes még megemlíteni a nyílt forráskódú megoldásokat, amelyeket futtathatnánk egyetemi hardveren. Ezzel garantálni tudnánk, hogy nem futunk ki abból a keretből, amit az Amazon ingyenesen nyújt minden új regisztrációhoz. A legnagyobb akadály, hogy éles környezetben jelenleg CIRCLE alapú Infrastructure-as-a-Service szolgáltatás érhető csak el a <https://cloud.bme.hu/> oldalon. Nem kis energiabefektetést igényelne további

szolgáltatások telepítése és konfigurálása úgy, hogy egyszerre ki tudják szolgálni a tárgy összes hallgatóját.

1. gyakorlat

Szolgáltatási rétegek

Amikor regisztráció után először bejelentkezel az AWS-be, akkor csak a bankkártyád egyenlege szab határt annak, hogy milyen szolgáltatásokat veszel igénybe. Legelőször ezért megismerkedünk az AWS felhasználó- és jogosultságkezelésével. Példát mutatunk arra, hogy egy hús-vér fejlesztő hozzáférését hogy tudjuk korlátozni, illetve hogy hogyan tudunk jogokat delegálni egy szoftvernek, ami további szolgáltatásokat vesz igénybe a kontónkra.

A gyakorlat második felében egy-egy egyszerű példát nézünk meg a három szolgáltatási rétegből:

1. Elindítunk egy virtuális gépet az Elastic Compute Cloud (EC2) *infrastruktúra-szolgáltatásban* (*Infrastructure-as-a-Service, IaaS*).
2. Felépítünk egy klasztert az Elastic Kubernetes Service (EKS) *platformszolgáltatásban* (*Platform-as-a-Service, PaaS*). Az ehhez szükséges parancssori segédprogramokat az előbb indított virtuális gépre fogjuk telepíteni, a saját gépeden csak egy SSH kliensre lesz szükség.
3. A klaszterre ezután telepítjük a Rocket.Chat *szoftverszolgáltatást* (*Software-as-a-Service, SaaS*), amellyel egy a Slackhez hasonló üzenetküldő szolgáltatást nyújtunk a felhasználóinknak.

1. Amazon Resource Name (ARN)

Az Amazon [jelenleg](#) a Föld 38 régiójából (*region*) szolgálja ki a felhőszolgáltatását. Ezeket kódnevek jelölik, a stockholmi adatközpontra például `eu-north-1`-ként hivatkozunk. Az igénybe vehető szolgáltatások gyűjtőfogalma az *AWS erőforrás* (*AWS resource*), amit a [dokumentáció](#) elég tágan úgy definiál, hogy „minden olyan entitás, amivel dolgozni tudsz”. AWS erőforrás egy virtuális gép, egy adatbázis tábla, de például a felhasználói fiók is, amin keresztül a többi szolgáltatást eléred.

Az egyes erőforrásokat az *Amazon Resource Name (ARN)* alapján különböztetjük meg. A 4. szakaszban megismerkedünk majd az EC2 IaaS szolgáltatással, amiben egy virtuális gép példányra így hivatkozhatunk:

```
arn:aws:ec2:eu-north-1:123456789012:instance/i-1234567890abcdef0
```

Az egyes részek jelentése:

arn Kötelező prefixum.

aws Esetünkben ez is fix lesz. Kínai régióknál itt `aws-cn` szerepel, az amerikai kormányzati régióknál pedig `aws-us-gov`. Európából ezekhez nincs hozzáférésünk¹.

¹Európából érdemes figyelemmel követnünk a 2025-ben induló [AWS European Sovereign Cloud](#) projektet.

ec2 Az igénybe vett szolgáltatás rövidítése.

eu-north-1 A régió, ahonnan a szolgáltatást igénybe vesszük, jelen esetben a már említett stockholmi adatközpont.

123456789012 Az AWS-ben minden felhasználónak van egy 12 számjegyből álló azonosítója. Egy AWS erőforrás ARN-jében ez a tulajdonosát jelöli.

instance Az AWS erőforrás típusa, jelen esetben egy virtuális gép példány.

i-1234567890abcdef0 Az AWS erőforrás azonosítója. Ennek nincs egységes formátuma, típustól függően változhat. Fontos észben tartani, hogy ez csak az ARN többi része által leszűkített kontextusban egyedi, vagyis az adott régióban, az adott felhasználó erőforrásai közül stb.

A későbbiekben hasznos lesz, hogy az ARN-ekben használhatunk helyettesítő (wildcard) karaktereket, amik erőforrások egy csoportját jelölik. Így hivatkozhatunk például egy felhasználó összes virtuális gépére egy adott régióban:

```
arn:aws:ec2:eu-north-1:123456789012:instance/*
```

2. Első lépések

2.1. Feladat: regisztráció és bejelentkezés az AWS-be

1. Az <https://aws.amazon.com/> oldalon kattints a **Create account** gombra!
2. Regisztrációkor alapértelmezetten egy úgynevezett root felhasználó jön létre. A 3.1. szakaszban beszélünk majd róla többet. Egyelőre az a lényeg, hogy egy működő e-mail címet adj meg. Adj nevet is a fióknak, majd kattints a **Verify email address** gombra!
3. Írd be az e-mailben kapott visszaigazoló kódot!
4. Most van lehetőségod megadni a root felhasználó jelszavát. Az AWS minden esetben két-faktoros azonosítást használ. Alapértelmezetten minden bejelentkezéskor egy kódot küld e-mailben, de ezt le lehet cserélni kényelmesebb azonosítási módokra.
5. A következő képernyőn válaszd az ingyenes díjsomagot! Ezzel a következő 6 hónapban ingyen igénybe vehetsz 100 dollárnyi szolgáltatást, amit [bizonyos feltételeket](#) teljesítve további 100 dollárral növelhetsz. Ha ezen túl is szeretnéd használni az AWS-t, akkor az ingyenes díjsomagról explicit át kell állnod a fizetősre, úgyhogy nem kell váratlan költségektől tartanod.

Sign up for AWS

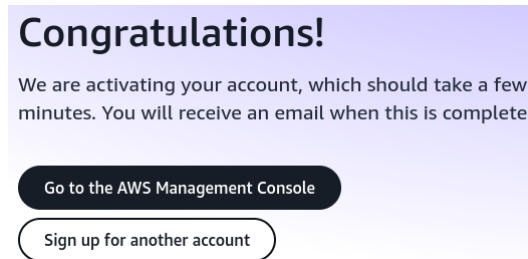
Choose your account plan

	
Free (6 months) Learn, experiment, and build prototypes	Paid Develop production-ready workloads
✓ Receive up to \$200 in credits	✓ Receive up to \$200 in credits
✓ Includes free usage of select services	✓ Includes free usage of select services
✗ Workloads scale beyond credit thresholds	✓ Workloads scale beyond credit thresholds
✗ Access to all AWS services and features	✓ Access to all AWS services and features
ⓘ After the 6 month free period or when all credits are used, you can choose to upgrade to a paid plan. Otherwise, your account closes automatically.	ⓘ After all of your credits are used, you are charged using pay-as-you-go pricing.
Choose free plan	Choose paid plan

6. A következő lépésekben sajnos viszonylag sok személyes adatot meg kell adni. Működő

telefonszámra van szükség, mert SMS-ben visszaigazoló kódot küldenek rá. Ugyanígy a bankkártyaadatok megadását sem lehet megúszni, 1 eurót zárol rajta az Amazon, amit pár napon belül feloldanak. Az ingyenes díjsomagban ennél több költségre nem kell számítanod.

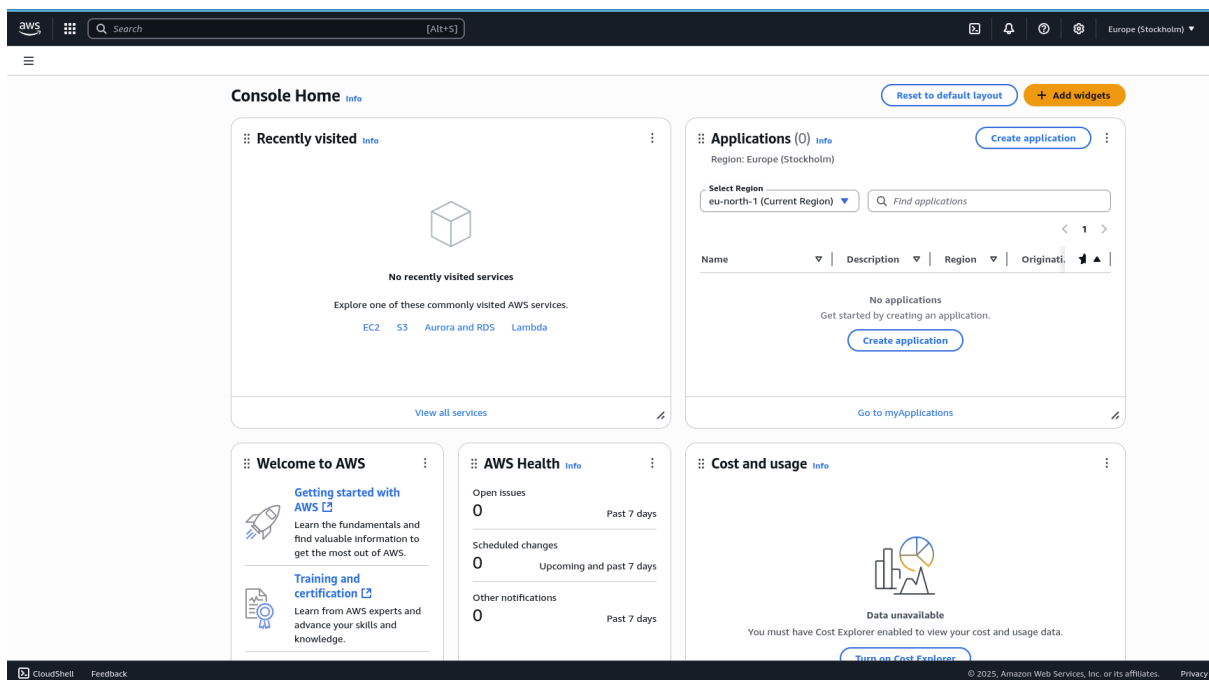
7. Ha minden jól megy, akkor a következő üzenet fogad:



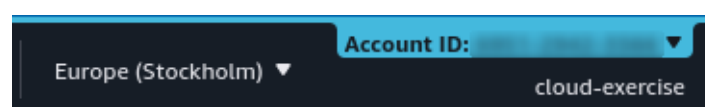
8. Amint megjön az ígért e-mail, kattints a [Go to the AWS Management Console](#) gombra, ami automatikusan bejelentkeztet.

2.2. AWS Management Console

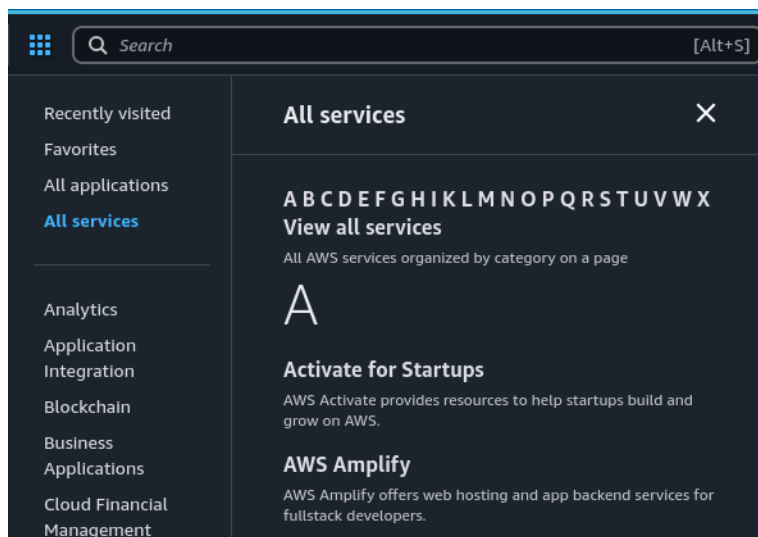
Bejelentkezés után az AWS fő webes felülete, a *Management Console* fogad majd. Itt különböző widgeteket találsz, amelyekkel gyorsan tájékozódhatsz az igénybe vett szolgáltatásokról és azok költségeiről. Ezeket személyre is szabhatod, átrendezheted őket, és újakat vehetsz fel.



A jobb felső sarokban találod a felhasználói fiókad adatait, mint a korábban említett 12 számjegyből álló azonosítót, és a régiót amiben éppen dolgozol.

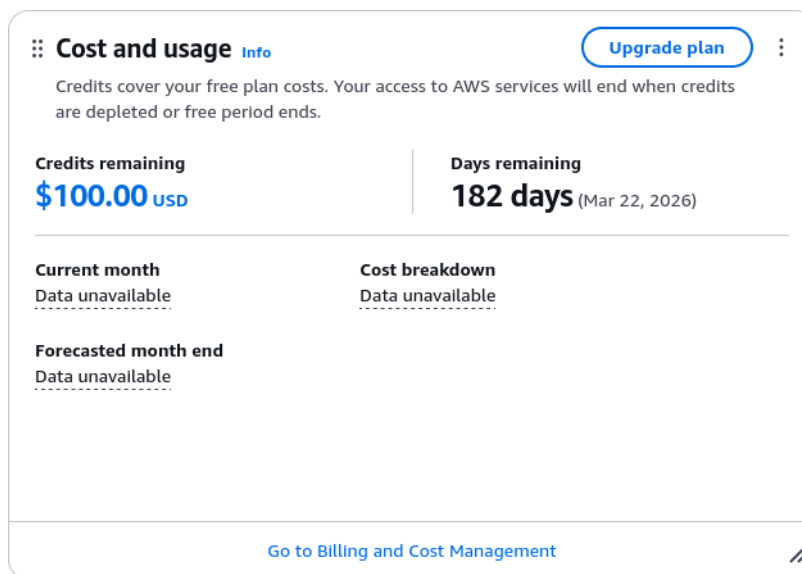


Számunkra legfontosabb a *szolgáltatás menü* a bal felső sarokban, ahonnan ABC sorrendben vagy kategóriákra bontva érheted el az összes szolgáltatást.



2.3. Feladat: kiadások követése a Management Console-on

1. A Cost and usage widgetben kattints a **Turn on Cost Explorer** gombra!
2. Átkerülsz a Billing and Cost Management home oldalra, de kattints a bal felső sarokban lévő AWS logóra, hogy vissza kerülj a Management Console-ra!
3. A Cost and usage widget megváltozott, mostantól mutatja, hogy mennyi van még vissza a 6 hónapos időkeretedből és a 100 dolláros pénzkeretedből.



3. AWS Identity and Access Management (IAM)

3.1. Alapfogalmak

Regisztráció után a *root felhasználó* fiókjába jelentkezünk be. Ennek hasonló a szerepe az azonos nevű fiókhoz UNIX-on vagy a rendszergazdáéhoz Windowson. Ez az a fiók, amelyikből minden

feladatot el tudunk végezni az AWS-ben (és [bizonyos feladatokat](#), mint például néhány számlázással kapcsolatos kérdést csak innen).

A továbbiakban a következő [alapfogalmakat](#) fogjuk használni (lásd még az [1.1.](#) ábrát):

IAM felhasználó (IAM user) Az AWS szolgáltatásokat igénybe vevő személyek vagy alkalmazások.

IAM csoport (IAM group) IAM felhasználók névvel ellátott halmaza.

IAM szerep (IAM role) Előfordulhat, hogy bizonyos feladatokat nem saját jogunkon, hanem egy IAM szerepet betöltve végezhetünk el. Ahhoz hasonló ez, mint amikor valakit az egyetem rektorává választanak, és ezzel plusz jogköröket kap, amiket a mandátuma lejártával elveszít.

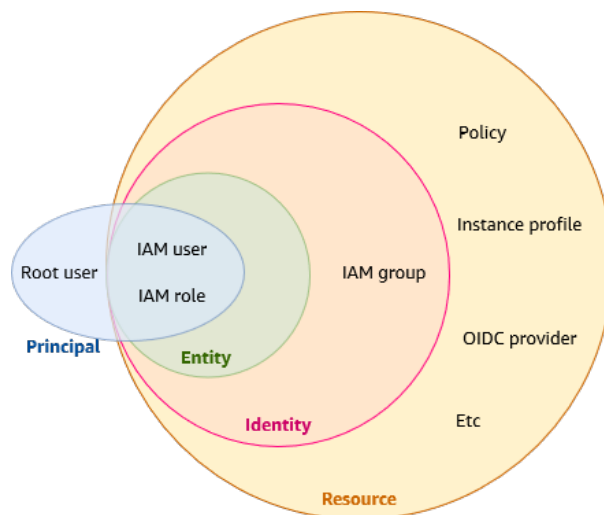
identitás Az IAM felhasználót, IAM csoportot és IAM szerepet magába foglaló gyűjtőfogalom.

alany (principal) Egy AWS erőforrás szemszögéből lényegtelen, hogy őt a root felhasználó, egy IAM felhasználó vagy éppen egy IAM szerepet felvett felhasználó éri el. Ezekben a helyzetekben használjuk rájuk az *alany* gyűjtőfogalmat.

IAM erőforrás (IAM resource) Az IAM-beli AWS erőforrások (lásd [1.](#) szakasz), mint az IAM felhasználók, IAM szerepek stb.

IAM szabályzat (policy) A jogosultságokat, vagyis hogy miket szabad és miket tilos megtenni, azokat *IAM szabályzatok* írják le. IAM szabályzatot elsősorban vagy egy IAM identitáshoz rendelünk (*identity-based policies*, pl. felhasználóként nekem mihez van jogosultságom) vagy egy AWS erőforráshoz (*resource-based policies*, pl. az alany mit csinálhat ezzel a virtuális géppel). Vannak egyéb, például szervezeti szintű szabályzatok, de ezekkel a továbbiakban nem foglalkozunk.

felügyelt szabályzat (managed policy) Az AWS előre [definiál](#) és karbantart egy sor szabályzatot. Ezek közül néhányat mi is használni fogunk. Fontos észben tartani, hogy ezeket bármikor módosíthatja az Amazon, nem maradnak az akkori állapotukban, amikor beállítottuk őket.

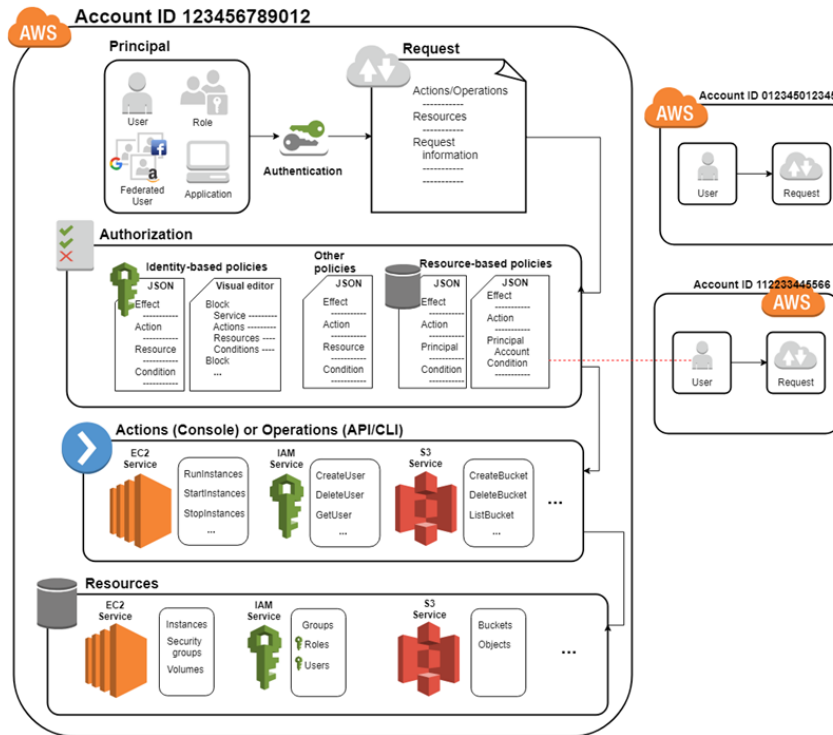


1.1. ábra. Az AWS IAM alapvető fogalmainak viszonya

3.2. Műveletek az AWS-ben

Ha valamit tenni szeretnénk az AWS-ben, azt mindig egy *kérés (request)* formájában kell megfogalmaznunk, ami a következőket tartalmazza:

- az elvégezni kívánt *művelet (action/operation)*², például virtuális gép indítása,
- az AWS erőforrás, amelyen a műveletet el akarjuk végezni,
- az alany, aki a műveletet el akarja végezni,
- a művelet paraméterei és környezeti változók.



1.2. ábra. Egy kérés menete az IAM-ben

Az 1.2. ábra mutatja egy kérés életciklusát. Először azonosítja magát az alany. Ezután a rendszer veszi az összes releváns IAM szabályzatot, vagyis ami a kérésben szereplő alanyra vonatkozik, ami az AWS erőforrásra vonatkozik és ami még a további szabályzatokból szóba jöhet. Ezek kiértékelésekor az úgynevezett *explicit elutasítás* elvét követi: ha csak egy is elutasítja az adott műveletet, akkor az egész kérést elutasítja. Ha valamiről esetleg nem szól egyik szabályzat sem, akkor alapértelmezetten minden elutasításra kerül. Vagyis amennyiben valamilyen jogot meg szeretnénk adni, azt vagy az alanyra vagy a AWS erőforrásra vonatkozó szabályzatban engedélyeznünk kell, és sehol máshol nem szabad megtiltanunk. A kérés csak ezt követően jut el a konkrét szolgáltatáshoz, amely elvégzi a kért műveletet.

Az AWS többi szolgáltatásának dokumentációja külön fejezetet szentel az IAM-mel való integrációnak. Ezek tartalmazzák, hogy az adott szolgáltatásban specifikusan milyen AWS erőforrások felügyelhetők IAM-mel, és ezekre milyen jogosultságokat állíthatunk be. A továbbiakban például az [EC2](#) és [EKS](#) dokumentációinak idevágó részeit vesszük alapul, de a konkrét feladatok előtt megismerkedünk a szolgáltatásokkal.

²Az angol nyelvű dokumentáció a Management Console-on végzett műveletekre az *action* szót használja, az API-n vagy parancssoron keresztül pedig az *operation*-t. Mivel számunkra nem fontos köztük különbséget tenni, mindkettőt műveletként fordítottuk.

4. Infrastruktúra-szolgáltatás: Elastic Compute Cloud (EC2)

Az AWS első szolgáltatásai közt volt az *Elastic Compute Cloud (EC2)*, amely lassan **két évtizede** érhető el a nyilvánosság számára. Virtuális gépek futtatása a legalapvetőbb funkciója, de felhőszolgáltatásról lévén szó ezeket nem csak kézzel tudjuk indítani. Létre tudunk hozni olyan klasztereket, amelyekben a virtuális gépek száma a terheléssel együtt változik (nő, és ugyanilyen fontos, hogy csökken is), amit részletesen monitorozni is tudunk. Ez az úgynevezett *horizontális skálázódás* a következő gyakorlat témája lesz majd.

A virtuális gép hardverét és szoftverét két külön listából választhatjuk ki. Előbbit nevezük a gép *példány típusának* (*instance type*). Ez minden esetben magába foglalja a (virtuális) processzormagok számát és a rendelkezésre álló memóriát. A gyakorlaton például egy `t3.micro` típusú gépet fogunk használni, ami 2 processzormagot és 1 GB memóriát tartalmaz. Ez természetesen az árát is meghatározza, ami esetünkben **kicsivel több mint óránként 1 cent**. Ezeken az általános célú gépeken kívül elérhetőek például olyanok, amik nagy memóriaigényű feladatokra vannak optimalizálva, vagy valamilyen extra hardvert pl. GPU-t vagy AI gyorsítót tartalmaznak. A különböző lehetőségek közt az [Instance Type Explorer](#) segít eligazodni.

A virtuális gépek háttértárát egy külön szolgáltatás, az [Amazon Elastic Block Store \(EBS\)](#) tárolja. Ez kifejezetten olyan lemezképekre van optimalizálva, amelyek egyetlen virtuális gépre vannak felcsatolva. A feladatoknál látni fogjuk, hogy szerencsére nem szükséges a lemezképet külön létrehoznunk, hanem a virtuális géppel egyszerre meg tudjuk tenni. Ha szeretnénk több virtuális gép között fájlokat megosztani, akkor **rengeteg** további szolgáltatás közül választhatunk, hálózati fájlrendszerektől kezdve objektumtárolókon keresztül egészen a specifikusan cache-elésre szánt gyors megoldásokig.

Azt, hogy a virtuális gép merevlemezére milyen szoftver kerül, azt az *Amazon Machine Image (AMI)* határozza meg. Vannak olyan lemezképek, amelyek csak az operációs rendszert tartalmazzák (elérhető Windows, macOS, és rengeteg Linux disztribúció), de vannak amelyekre valamilyen további szoftver is telepítve van. Természetesen az opciókat szűkíti a választott hardver, ARM processzorra ne válasszunk x64-es operációs rendszert, és fél GB RAM-on sem éri meg Windowszal próbálkozni. A szoftverlicencek árát sajnos áthárítja ránk az Amazon, úgyhogy a példány típusok mellett több árát is látni fogunk.

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour On-Demand RHEL base pricing: 0.0396 USD per Hour

On-Demand SUSE base pricing: 0.0108 USD per Hour On-Demand Linux base pricing: 0.0108 USD per Hour

On-Demand Windows base pricing: 0.02 USD per Hour

All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

4.1. A virtuális gép személyre szabása

SSH hozzáférés

Az Amazon Linux 2023 AMI-t fogjuk használni, ami egy általános célú Linux disztribúció az Amazontól. Minden Amazon Linuxot futtató virtuális gépen automatikusan létrejön egy `ec2-user` nevű felhasználói fiók, amibe SSH klienssel tudunk bejelentkezni. Biztonsági okokból a jelszóval történő autentikáció alapértelmezetten le van tiltva (és utólag sem ajánlott engedélyezni), helyette egy kulcspárral tudunk bejelentkezni: a saját gépeden az SSH kliensnek megadod a privát kulcsot, a virtuális géphez pedig létrehozásakor hozzárendeljük a publikus kulcsot, amit továbbít a rendszer a rá telepített SSH szervernek.

Kontextualizáció: cloud-init

Az AWS a Canonical [cloud-init](#)jét használja a virtuális gépek kontextualizálására. Ez egy olyan háttérfolyamat a gépen, amely átveszi az AWS-től azokat a paramétereket, amelyeket a webes felületen megadtunk, és ezeknek megfelelően szabja személyre a példányt. Az ő dolga például az előbb említett SSH kulcs telepítése, vagy ha létrehozáskor megadtunk egy hálózati fájlrendszert, akkor annak felcsatolása. Természetesen a webes felület nem tud opciót nyújtani minden felmerülő igényre, úgyhogy a *user data* mezőben megadhatunk egy shell szkriptet³, amit első indításkor lefuttat a cloud-init.

Mi ezt a funkciót szoftverek telepítésére fogjuk használni. Bonyolultabb esetekben robusztusabb megoldást jelentenek erre a feladatra az Infrastructure-as-Code (IaC) eszközök, mint például az [AWS CloudFormation](#), amiről a 3. gyakorlaton lesz majd szó.

További AWS erőforrások elérése a virtuális gépből

Nemsokára egy EC2-ben indított virtuális gépre fogjuk telepíteni azokat a parancssori eszközöket, amelyekkel aztán az AWS egyik platformszolgáltatását használjuk. A parancssori eszközöknek autentikálniuk kell magukat az IAM-ben minden művelet előtt, úgyhogy **telepíteni kéne** melléjük a szükséges hitelesítő adatokat. Szerencsére ezt egy EC2 példánynál meg tudjuk spórolni, ugyanis hozzá tudunk rendelni egy IAM szerepet. Innentől automatikusan ez a szerep lesz minden olyan művelet alanya, amit a virtuális gépről indítunk. Annyi megkötés van, hogy az IAM szerep létrehozásakor meg kell adnunk, hogy az EC2-ben fogjuk használni. Ekkor létrejön egy a szerephez rendelt és vele megegyező nevű *példány profil* (*instance profile*), és szigorúan véve ez az, amit a virtuális géphez rendelünk.

Érdemes még pár szót ejteni az [AWS CloudShell](#)ról. Ezt a webes felületen bárhonnan elérheted a képernyő bal alsó sarkából. Ahogy a nevéből már talán kitaláltad, ez is parancssoros hozzáférést ad a felhőszolgáltatásokhoz. A háttérben ugyanúgy egy Amazon Linux 2023-as gép indul, amire egy webes SSH klienssel automatikusan bejelentkezik a rendszer. Legnagyobb hátránya, hogy munkamenetek közt csak a `$HOME` könyvtár tartalma marad meg garantáltan, a rendszerkönyvtárak és így a telepített szoftverek [nem feltétlenül](#). Ezért egyszeri feladatokra tökéletes, de nekünk a következő gyakorlaton újra fel kéne rá telepítenünk mindent.

```
CloudShell
eu-north-1 +
~ $ neofetch

            _/oydNNdyo:.\
          .:~shmmmmmmmmmmmmmmhms+:.\
        +-hNNNNNNNNNNNNNNNNNNNNNNNNNNNNho-
        .:/~shmmmmmmmmmmmmmmhms+/-
dNmhs+:.\:/oo:/.\          .:~shmNd
dmmmmmmmmmmmmmmNdhs+:.\      .:~shdNNNNNNNNNNNNNd
odNNNNNNNNNNNNNNNNNNNNNNNd
dmmmmmmmmmmmmmmmmmmmmNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmmmNd
dmmmmmmmmmmmmmmmmmmmmNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmmmNd
dmmmmmmmmmmmmmmmmmmmmNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmmmNd
dmmmmmmmmmmmmmmmmmmmmNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmmmNd
dmmmmmmmmmmmmmmmmmmmmNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmmmNd
dmmmmmmmmmmmmmmmmmmmmNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmmmNd
.:~ydNNNNNNNNNNNNNNNNNNNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmNd
.:~shNNNNNNNNNNNNNNNNNNNd    ymmmmmmmmmmmmmmmmmmmmNd
ymmmmmmmmmmmmmmmmmmmNd
~ $
```

³A shell szkript mellett van még pár **további** fájltypus, amiket kezelni tud a cloud-init.

5. Szoftverszolgáltatás: Rocket.Chat

A következőkben a [Rocket.Chat](#) nyílt forráskódú üzenetküldő szoftvert fogjuk példának használni. Saját infrastruktúrára telepítve Discordhoz vagy Slackhez hasonló szolgáltatást tudunk vele nyújtani. Az adatok feletti teljes kontroll érdekében például az amerikai hadsereg és légierő saját szerverparkon futtatja, mi viszont most a felhőbe fogjuk telepíteni.

5.1. Mikroszolgáltatás architektúra

A Rocket.Chat telepíthető mikroszolgáltatás architektúrában. Ennek lényege, hogy az alkalmazásunkat több lazán kapcsolt, kevés felelősséggel bíró szolgáltatásra bontjuk, amelyek valamilyen egyszerű protokollon keresztül kommunikálnak egymással. Most nem célunk elmélyedni a témában, csak megemlítjük, hogy a telepítés után milyen mikroszolgáltatásokat fogunk látni:

Rocket.Chat monolith A szoftver képes monolitikus szolgáltatásként is futni, ebben az esetben minden feladatot ez a monolit lát el. Mikroszolgáltatásos telepítés esetén a monolit megmarad, de minden kiszervezett feladat le van benne tiltva.

Authentication A felhasználói fiókok autentikációjáért felelős.

Authorization A jogosultságkezelésért felelős.

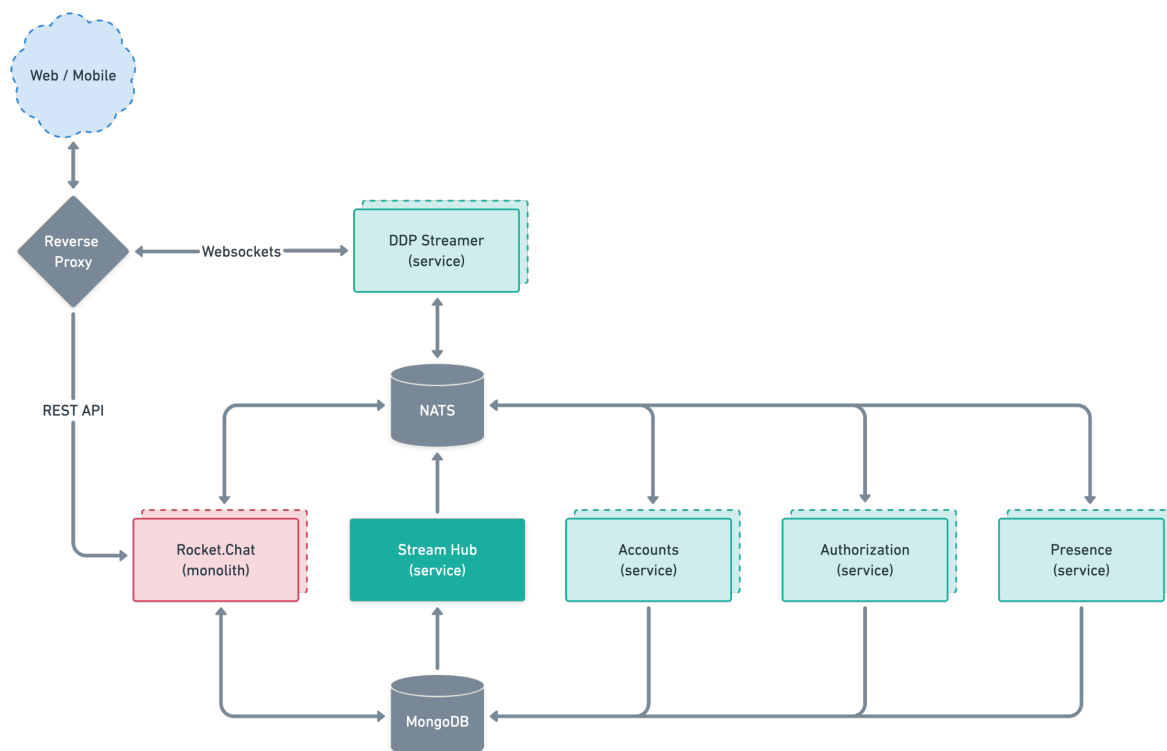
Presence A felhasználók státuszait (online, elfoglalt, távol) és státuszváltozásait kezeli.

MongoDB NoSQL adatbázis. Kizárólag ezt használja perzisztens adattárolásra, nem használ pl. blokkos tárolást.

DDP Streamer A WebSocket kapcsolatokat kezeli [Distributed Data Protocol \(DDP\)](#) felhasználásával.

NATS A többi mikroszolgáltatás kommunikációját lehetővé tevő üzenet sín (message bus).

Stream Hub Üzenetbróker, ami a MongoDB-ből érkező eseményeket továbbítja a többi mikroszolgáltatás felé.



6. Platformszolgáltatás: Elastic Kubernetes Service

6.1. Alkalmazáskonténerek

Minden programot valamilyen *alkalmazás platformra* építünk. Ez magába foglalja a választott programozási nyelv fordítóját vagy interpreterét és a gyakori feladatokat megvalósító szabványos könyvtárat. Beágyazott rendszereken ez általában egy C fordítót és a szabványos C könyvtárat jelenti, Windowson a Visual C++-t, a szabványos C++ könyvtárat és a Win32 API-t, webes környezetben pedig például a Node.js-t, ami magába foglalja a V8 JavaScript motorra épülő interpretert és a függvénykönyvtárt is.

Egy új Node.js verzióval változik a platform, amihez a rá épített programoknak alkalmazkodniuk kell. Ennek tempója projektenként változó, ami egy felhőszolgáltatás több bérlős modelljében ahhoz a problémához vezet, hogy egy adott pillanatban minden egyes platform több különböző verzióját támogatni kéne. Erre nyújtanak megoldást az *alkalmazáskonténerek*, amik egybe csomagolják a programot a futtatásához szükséges platformmal. Ezzel a fejlesztő felelősségévé válik, hogy a platformot naprakészen tartsa, a felhőszolgáltatónak elég az operációs rendszert szolgáltatnia. A gyakorlaton most ennél mélyebbre nem megyünk, de az alkalmazáskonténerek technikai részleteire és konkrétan a [Docker](#) konténerplatformra előadáson visszatérünk majd.

6.2. Konténer orkesztráció

Minden mikroszolgáltatást külön konténerbe teszünk, és virtuális gép(ek)en futtatjuk őket. Amíg kevés felhasználónk van, addig elég minden mikroszolgáltatásból egyet indítani, és a jobb kihasználás érdekében futtathatunk több konténert ugyanazon a virtuális gépen. Minden sikeres szolgáltatás életében eljön viszont az a pillanat, amikor már túl hosszú a válaszidő, ha mindenkinek egyetlen példány szolgál ki. Az infrastruktúra-szolgáltatáshoz hasonlóan itt is a horizontális skálázódás a megoldás, vagyis a leterhelt konténerekből több példányt indítunk. Ezt a párhuzamosítást természetesen magának a szoftvernek is támogatnia kell, és a Rocket.Chat fel is van készítve ilyen működésre. A terhelés növekedésével az EC2 is képes automatikusan horizontálisan skálázódni, vagyis újabb virtuális gépeket indítva egyre nagyobb klasztert rakni a konténereink alá.

*Konténer orkesztrációnak*⁴ hívjuk azt a feladatot, amikor el kell döntenünk, hogy melyik konténert melyik virtuális gépre rakjuk, és hogy esetleg mozgassunk-e át egy már futó konténert egyik gépről a másikra. Természetesen ezt nem kézzel csináljuk, hanem szoftverrel automatizáljuk. Kezdetben minden felhőszolgáltató saját megoldást dolgozott ki, az Amazon az [Elastic Container Service \(ECS\)](#) formájában. A Google nyílt forráskódú [Kubernetes](#) nevű megoldása viszont kvázi iparági szabvánnyá vált, így az Amazon is elindította a saját [Elastic Kubernetes Service \(EKS\)](#) szolgáltatását. Mi is ezt fogjuk ma használni.

Application Load Balancer (ALB) Egy Kubernetes klaszteren a *belépési szabályzat (ingress)* határozza meg, hogy egyes kérések a hálózatról melyik konténerhez jutnak el. Ennek segítségével tudjuk például megoldani a terheléelosztás problémáját: minden beérkező kérésről el kell döntenünk, hogy egy mikroszolgáltatás több éppen futó példánya közül melyik szolgálja ki. Erre majd egy későbbi gyakorlaton fogunk visszatérni, ma csak telepítjük a szükséges szoftvereket.

A belépési szabályzat alapján a csomagok tényleges irányítását a *belépésvezérlő (ingress controller)* végzi. Az implementációk közül érdemes megemlíteni az nginx-et, ami nyílt forráskódú szoftver, és tetszőleges Kubernetes klaszterre telepíthető. Ezen kívül az összes szolgáltatónak van

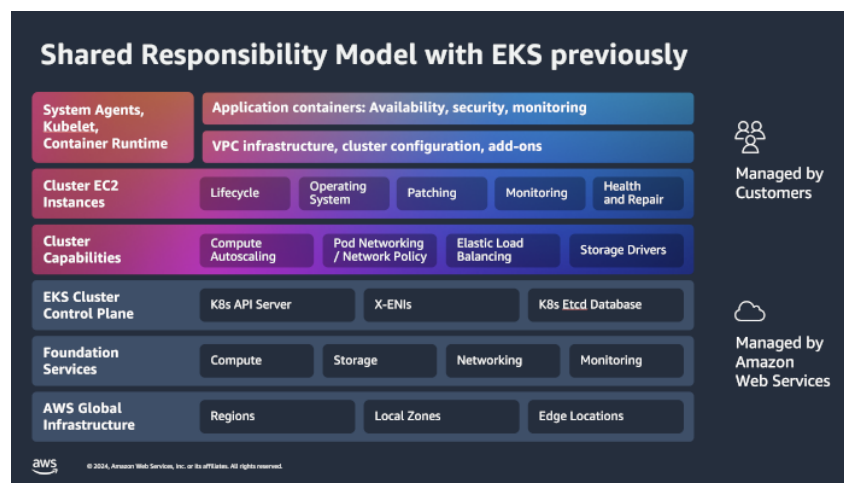
⁴Az eredeti angol *orchestration* kifejezés szó szerint hangszerezést jelent, vagyis egy zenekarban az egyes szőlők hangszerekhez rendelését. Bár ez egy elég kifejező fordítás lenne, a magyar szaknyelvbe inkább az orkesztráció jövevényszó épült be.

saját implementációja is, az Amazoné az *Application Load Balancer (ABS)*, amit mi is használni fogunk.

Perzisztens tárolás A Kubernetesen futó alkalmazások az adataikat a *tároló osztályokon (storage class)* keresztül tárolják. A korábban megismert Elastic Block Store (EBS) szolgáltatásban fogunk létrehozni egy kötetet, ami alapértelmezett tároló osztálynak beállítva az összes adatot tárolni fogja.

6.3. EKS Auto Mode

Egy Kubernetes klaszter telepítésében és üzemeltetésében az egyik legnagyobb kihívás, hogy nem csak a konténerek számának kell automatikusan skálázódnia, hanem alacsonyabb szinten a virtuális gépek számának is. Ezt könnyíti meg az EKS Auto Mode, amivel az Amazon átvesszi az infrastruktúra irányításának felelősségét. A klaszter alatt automatikusan létrejönnek a virtuális gépek, és a későbbiekben is beavatkozás nélkül változik a számuk a terhelés függvényében.



(a) Hagyományosan



(b) Auto Mode-ban

1.3. ábra. A *felelőségek megoszlása* közted és az Amazon közt.

6.4. Parancssori eszközök: aws, eksctl, kubectl és Helm

Az AWS szolgáltatások parancssorból az AWS CLI segítségével érhetőek el. Az **aws** parancs telepítve van az Amazon Linuxot futtató virtuális gépekre. Ezt sokszor nem is közvetlenül használjuk, hanem más parancsok hívják meg a háttérben.

Erre példa az **eksctl**, amivel klasztereket lehet létrehozni, törölni, és a beállításait módosítani. A nevéből talán már rájöttél, hogy ez egy Amazon EKS specifikus eszköz, ezeket a feladatokat más szolgáltatóknál másik eszközzel kell elvégezni.

Miután létrehoztuk a Kubernetes klasztert, a nyílt forráskódú **kubectl** eszközzel tudjuk beállítani többek közt az ingress és a tároló osztályt. Ez a szoftver már szolgáltatófüggetlen, ugyanúgy használjuk más szolgáltatóknál futó Kubernetes klasztereken vagy például a fejlesztési célra a saját gépünkre telepített minikube klaszteren.

Szerencsére nem kell kézzel telepítenünk minden egyes mikroszolgáltatást a Kubernetesre, hanem használhatjuk a **Helm** csomagkezelőt. Egy Helm chart nevű dokumentum írja le neki, hogy milyen lépéseket végezzen egy adott szoftver telepítésekor. Mi a Rocket.Chat fejlesztőinek hivatalos Helm chartját fogjuk használni.

7. Feladatok

7.1. IAM

Az első dolgunk annak a felhasználónak a létrehozása, aki majd elindítja a virtuális gépet EC2-ben.

- A szolgáltatás menüből válaszd az IAM-et!
- Az **Access management** > **Users** menüpontban kattints a **Create user** gombra!
- Add az új felhasználónak a **developer** nevet! A jelölőnégyzet kipipálásával adj neki hozzáférést a Management Console-hoz! Bár nem ajánlott, mégis válaszd csak az IAM felhasználó létrehozását! A jelszó megadása után kattints a **Next** gombra!

- Válaszd az `Attach policies directly` lehetőséget; keresd ki, és jelöld be az `AmazonEC2FullAccess` és az `IAMFullAccess` szabályzatokat; végül kattints a `Next`-re!
- Ha minden stimmel az utolsó oldalon, akkor kattints a `Create user` gombra!

A következő dolgunk létrehozni azt a szerepet, amit felvéve a virtuális gépben futó AWS CLI létre tudja hozni az EKS klasztert. Az `eksctl` [dokumentációjából](#) viszont megtudhatjuk, hogy ehhez nem csak az `AmazonEC2FullAccess` és az `AWSCloudFormationFullAccess` felügyelt szabályzatokat kell beállítanunk úgy, ahogy előbb a `developer` felhasználónál tettük, hanem nekünk kell létrehoznunk az `EksAllAccess` és az `IamLimitedAccess` szabályzatokat. Ennek lépései mindkét esetben a következők:

1. Az `Access management` `Policies` menüpontban kattints a `Create policy` gombra!
2. Felül válaszd a `JSON` opciót!
3. A szerkesztőbe illeszd be a szabályzat ajánlott tartalmát az `eksctl` [dokumentációjából](#)!
4. Az `<account_id>` sztringet mindenhol cseréld le a saját 12 jegyű azonosítóra, amit egyszerűen ki tudsz másolni a jobb felső sarokban az `Account ID`-ra kattintva lenyíló menüből. Kattints a `Next`-re!
5. Add meg a szabályzat nevét!
6. Kattints a `Create policy` gombra!

Most már semmi akadály, hogy létrehozzuk a szükséges szerepet:

1. Az `Access management` `Roles` menüpontban kattints a `Create role` gombra!
2. A `Trusted entity type` maradjon az alapértelmezetten kiválasztott AWS service. A `Service or use case` listából válaszd is ki az EC2-t, a szolgáltatást amiből egy erőforrás majd fel fogja venni a szerepet. Ezután kattints a `Next` gombra!
3. Keresd ki és jelöld be a következő szabályzatokat:
 - `AmazonEC2FullAccess`
 - `AWSCloudFormationFullAccess`
 - `EksAllAccess`
 - `IamLimitedAccess`

Ezután kattints a `Next` gombra!

4. Add a szabályzatnak az `EKS-poweruser-role` nevet!
5. Kattints a `Create role` gombra!

7.2. Átjelentkezés a developer fiókba

A további feladatokat már nem szükséges root felhasználóként megoldanunk.

1. Kattints a jobb felső sarokban az `Account ID` gombra!
2. Tedd a vágólapra a fiókod 12 számjegyű azonosítóját!
3. Kattints a `Sign out` gombra!
4. Fölül kattints a `Sign in to console` gombra!
5. Illeszd be a fiókod 12 jegyű azonosítóját, írd be a `developer` felhasználónevet és a jelszót, amit megadtál neki, majd kattints a `Sign in` gombra!

Érdemes megfigyelni, hogy bár távolról a Management Console nagyon hasonlít arra amit a root felhasználónál láttunk, most tele van hibaüzenetekkel. Ez nem véletlen, ugyanis a `developer` felhasználónak csak az EC2-höz adtunk hozzáférést.

7.3. SSH hozzáférés

Válaszd az EC2-t a szolgáltatás menüből! Az első dolgunk létrehozni az SSH hozzáféréshez szükséges kulcspárt. Erre [két](#) mód is van: vagy 1. az Amazon generálja őket, letöltöd, és importálod

a privát kulcsot a kliensedbe, vagy 2. létrehozol egy újat, vagy használsz egy meglévőt a saját gépedről, és a publikus kulcsot megadod az Amazonnak.

Új SSH kulcspár létrehozása

1. A **Network & Security** > **Key Pairs** menüpontban kattints a **Create key pair** gombra!
2. A Name mezőben add meg, hogy milyen néven szeretnél később hivatkozni a kulcspárra!
3. A kulcspár típusának válaszd az ED25519-et! A felajánlott opciók közül ez a modernebb és biztonságosabb, de már bő egy évtizede támogatja minden SSH kliens.
4. Válaszd ki a használni kívánt SSH klienshez tartozó fájlformátumot! A PuTTY-val való csatlakozás lépéseit
5. A kulcspár is egy AWS erőforrás, amit *címkékkel (tag)* láthatnánk el. Az címkék segítségével [tovább finomíthatjuk](#) a kulcspárhoz való hozzáférést az IAM-ben, de ezt a funkcionalitást most nem fogjuk használni.

Create key pair [Info](#)

Key pair
A key pair, consisting of a private key and a public key, is a set of security credentials that you use to prove your identity when connecting to an instance.

Name
cli-vm-key
The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type [Info](#)
☐ RSA ☒ ED25519

Private key file format
☒ .pem
For use with OpenSSH
☐ .ppk
For use with PuTTY

Tags - optional
No tags associated with the resource.
[Add new tag](#)
You can add up to 50 more tags.

[Cancel](#) [Create key pair](#)

6. Kattints a **Create key pair** gombra!
7. A privát kulcsot automatikusan letölti a böngésződ. **Fontos**, hogy az Amazon csak a kulcspár egyik felét, a publikus kulcsot tárolja. A privát kulcsot kizárólag most van lehetőség elmenteni, később nem tudod letölteni.

Meglévő SSH kulcs importálása

Ha ezt a lehetőséget választod, akkor feltételezzük, hogy már van egy SSH kulcspárod, vagy tudod hogy kell létrehozni.

1. A **Network & Security** > **Key Pairs** menüpontban kattints az **Actions** > **Import key pair** gombra!
2. A Name mezőben add meg, hogy milyen néven szeretnél később hivatkozni a kulcspárra!

3. Töltsd fel a publikus kulcsot tartalmazó fájlt, vagy illeszd a tartalmát a beviteli mezőbe!
4. A kulcspár is egy AWS erőforrás, amit *címkékkel (tag)* láthatnánk el. Az címkék segítségével [tovább finomíthatjuk](#) a kulcspárhoz való hozzáférést az IAM-ben, de ezt a funkcionalitást most nem fogjuk használni.

Import key pair Info

You can use a third-party tool to create your key pair, and then import the public key to Amazon EC2.

Import settings

Name

cli-vm-key

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair file

 Browse

Choose **Browse** and navigate to your public key. You may change the name of your key. Alternatively, paste the contents of your public key into the **Public key contents** text box.

 id_ed25519.pub
0.098kb

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAecaOm+  
+tVQSYoY14pXr+8JCOLXVYVtGOWYweAKHdd6 ldmnyblzs@fedora
```

Tags - optional

No tags associated with the resource.

 Add new tag

You can add up to 50 more tags.

Cancel

Import key pair

5. Kattints a **Import key pair** gombra!

7.4. Virtuális gép indítása

1. Az **Instances** » **Instances** menüpontban kattints a **Launch instances** gombra!
2. A Name mezőben adj nevet a virtuális gépnek!
3. Az alapértelmezett beállítások meg fognak felelni, viszont pár dolgot érdemes megfigyelni:
 - Amazon Linux 2023-at fogunk futtatni egy jól ismert 64 bites Intel CPU-n. Ebből a dobozból a Username mezőt érdemes még figyelni, innen tudhatjuk meg, hogy a gépre **ec2-user** néven fogunk tudni bejelentkezni.

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Quick Start

Amazon Linux
aws

macOS
Mac

Ubuntu
ubuntu

Windows
Microsoft

Red Hat
Red Hat

SUSE Linux
SUSE

Debian
debian


[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
ami-043339ea831b48099 (64-bit (x86), uefi-preferred) / ami-0d5eee4c9faade005 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.8.20250915.0 x86_64 HVM kernel-6.1

Architecture	Boot mode	AMI ID	Publish Date	Username	
64-bit (x86)	uefi-preferred	ami-043339ea831b48099	2025-09-10	ec2-user	Verified provider

- Egy t3.micro típusú gépet indítunk, ami 2 CPU magot és 1 GB memóriát tartalmaz.

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro
Family: t3 2 vCPU 1 GiB Memory Current generation: true
On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour On-Demand RHEL base pricing: 0.0396 USD per Hour
On-Demand SUSE base pricing: 0.0108 USD per Hour On-Demand Linux base pricing: 0.0108 USD per Hour
On-Demand Windows base pricing: 0.02 USD per Hour

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

4. A Key pair dobozban válaszd ki az előző lépésben létrehozott kulcspárt a listából!
5. A hálózati és háttértár beállításokat szintén elfogadjuk úgy ahogy vannak, de itt is érdemes megfigyelni a részleteket.
6. Az Advanced details dobozban válaszd az EKS-poweruser-role-t az IAM instance profile listából!
7. Az oldal aljára görgetve a User data pontban töltsd föl a user-data.sh fájlt!
8. Kattints a [Launch instance](#) gombra! Egy kis idő után valami hasonlót kell látnod:

✓ Success
Successfully initiated launch of Instance [\(i-0f10fe91308340af6\)](#)

7.5. Bejelentkezés a PuTTY SSH klienssel

1. Kattints a létrehozott példány kódjára, hogy megnézd a részleteit!
2. Másold az Auto-assigned IP address alatt látható címet a PuTTY Host name (or IP address) mezőjébe!

3. A **Connection** **SSH** **Auth** **Credentials** menüpontban a Private key for authentication alatt válaszd ki a privát kulcsot tartalmazó .ppk fájlt!
4. Kattints az **Open** gombra!
5. A login as: promptra írd be az ec2-user felhasználónevet!

Mielőtt tovább mennénk, érdemes ellenőrizni, hogy minden a helyén van-e. Add ki egymás után a következő parancsokat, és győződj meg róla, hogy nem hibaüzenetet kapsz:

```
aws --version
eksctl version
kubectl version
helm version
```

7.6. Kubernetes klaszter létrehozása Auto Mode-ban

1. Add ki a következő parancsot egy EKS klaszter létrehozásához Auto Mode-ban az eu-north-1 régióban:

```
eksctl create cluster --region=eu-north-1 --name=rocket-chat --enable-auto-mode
```

2. Menj el meginni egy kávé, mert ez pár percet igénybe fog venni!
3. A kubectl egy úgynevezett kubeconfig konfigurációs fájlból olvassa be a klaszter adatait. Add ki a következő parancsot, ami legenerálja a szükséges konfigurációs fájlt a frissen létrehozott klaszter adataival:

```
aws eks update-kubeconfig --region eu-north-1 --name rocket-chat
```

7.7. A klaszter konfigurálása és a Rocket.Chat telepítése

1. Töltsd le és csomagold ki a konfigurációs fájlokat a következő két parancssal:

```
curl http://vm.fured.cloud.bme.hu:16975/gyak1.tar.gz --output gyak1.tar.gz
tar -xf gyak1.tar.gz
```

2. Írd be a következő parancsokat egymás után! Ezekkel a letöltött konfigurációs fájlokból létrejön a szükséges storage class és ingress controller.

```
kubectl apply -f storage-class.yaml
kubectl apply -f ingress-class-params.yaml
kubectl apply -f ingress-class.yaml
kubectl apply -f ingress.yaml
helm repo add rocketchat https://rocketchat.github.io/helm-charts
helm install rocketchat -f rocket-chat.yaml rocketchat/rocketchat
```

3. Add ki a kubectl get ingress! Valami hasonlót kell kapnod, az (URL1) és (URL2) helyén hosszú URL-ekkel:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
rocket-chat-ingress	alb	*	(URL1)	80	6m10s
rocketchat-rocketchat	alb	(URL1)	(URL2)	80	3m17s

4. Egy tetszőleges szerkesztővel (nano, vim) írd át a rocket-chat.yaml-ben a host sorban szereplő URL-t az előző kimenetben kapott (URL2)-re!

5. Add ki a következő parancsot:

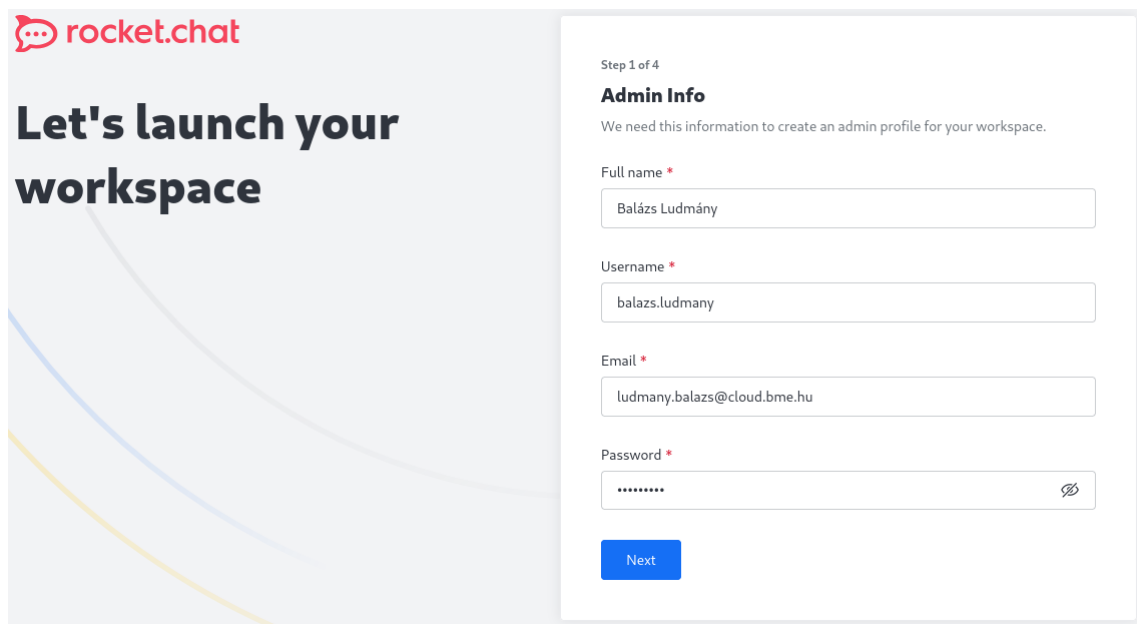
```
helm upgrade rocketchat -f rocket-chat.yaml rocketchat/rocketchat
```

A `kubectl get pods` paranccsal megnézhetjük a futó mikroszolgáltatásokat:

NAME	READY	STATUS	RESTARTS	AGE
rocketchat-account-6dfb598d47-6c4k7	1/1	Running	0	21m
rocketchat-authorization-5bb67895c7-85wzf	1/1	Running	0	21m
rocketchat-ddp-streamer-7787b5b97c-xbkvn	1/1	Running	0	21m
rocketchat-mongodb-0	2/2	Running	0	21m
rocketchat-nats-0	3/3	Running	0	21m
rocketchat-nats-1	3/3	Running	0	21m
rocketchat-nats-box-ddf65499c-fcvjv	1/1	Running	0	21m
rocketchat-presence-7b484bc575-vbvt	1/1	Running	0	21m
rocketchat-rocketchat-548bd64c6-jccwj	1/1	Running	0	18m
rocketchat-stream-hub-7594884864-g4zv4	1/1	Running	0	21m

7.8. A Rocket.Chat indítása

Ha ki szeretnéd próbálni a frissen telepített SaaS alkalmazásod, akkor menj végig a telepítő varázslón!



Step 1 of 4

Admin Info

We need this information to create an admin profile for your workspace.

Full name *

Balázs Ludmány

Username *

balazs.ludmany

Email *

ludmany.balazs@cloud.bme.hu

Password *

Next

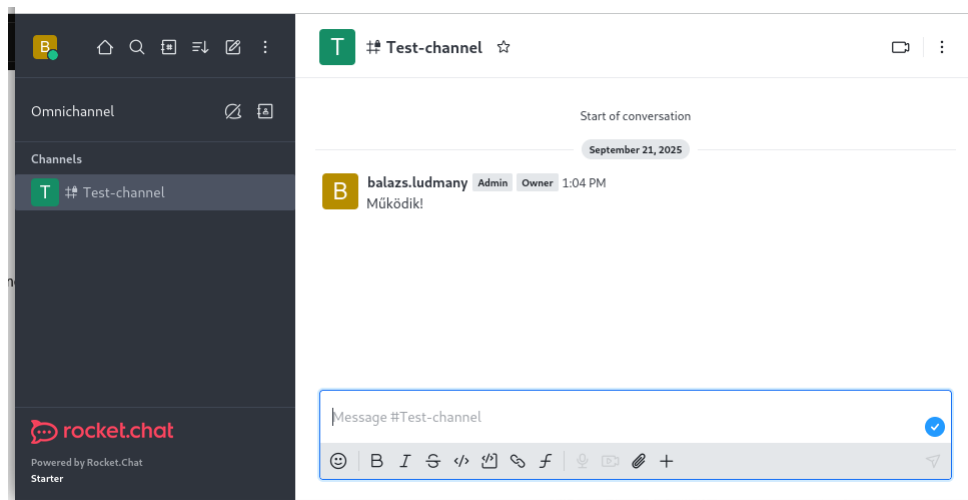
Az üzenetküldő felület pedig így néz ki:

7.9. A klaszter eltávolítása

Egy Kubernetes klasztert sajnos nem lehet „elaltatni”, hanem el kell távolítani, és legközelebb újból létrehozni. A következő parancs leállítja és törli a klaszter alatt futó virtuális gépeket is:

```
eksctl delete cluster --region=eu-north-1 --name=rocket-chat
```

A parancssoros eléréshez használt gépet leállításához pedig az EC2-ben az **Instances** menüpontban pipáld ki az előtte lévő jelölőnégyzetet, és kattints az **Instance state** gombra!



2. gyakorlat

Automatikus skálázás

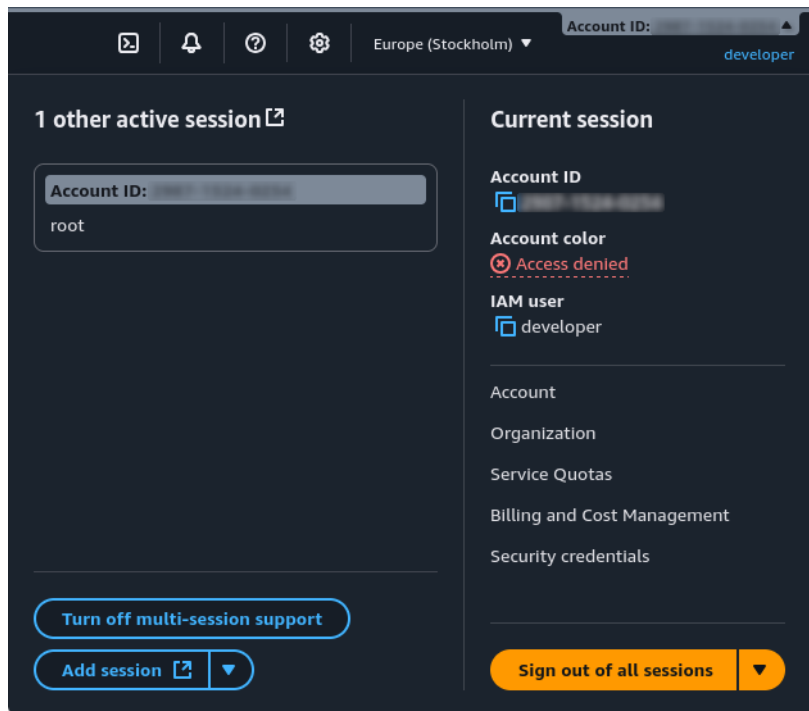
Az 1. gyakorlaton egyfajta „szélességi bejárást” tettünk a felhőrendszerek különböző területein. Foglalkoztunk mindhárom szolgáltatási réteggel, de a rendelkezésre álló időben nem jutottunk egyikben sem túl mélyre. Ennek következtében a legtöbb helyen az alapbeállításokat vagy előre kiadott szkripteket és konfigurációs fájlokat használtunk. A további gyakorlatok célja ezzel szemben a platformszolgáltatások „mélységi bejárása”. A 2. gyakorlaton először visszatérünk a Kubernetes klaszter konfigurálásának részleteire, és megismerkedünk az erre használt YAML nyelvvel. A hátralévő időben az automatikus horizontális skálázással foglalkozunk, amit a Locust keretrendszer segítségével terheléspróbának is alávetünk.

1. Klaszter konfigurálása részletesebben

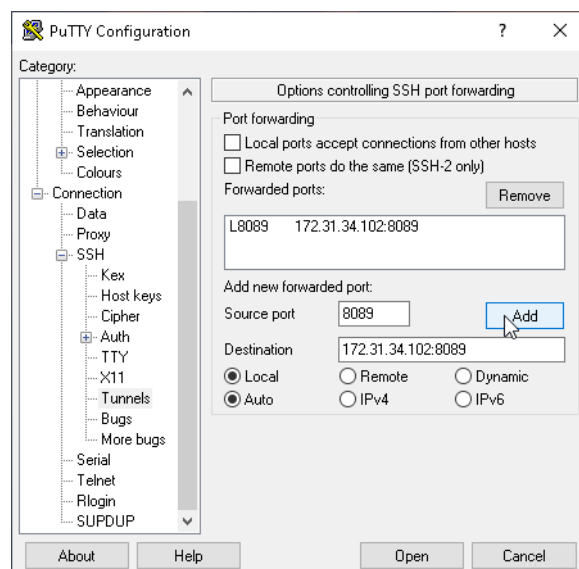
1.1. Feladat: a Kubernetes klaszter újraindítása

Az előző gyakorlat legvégén leállítottuk az adminisztrációra használt virtuális gépet és a klasztert. Mivel viszonylag sok időbe telik elindítani,

1. Jelentkezz be a Management Console-ba az <https://aws.amazon.com>-on. A későbbiekben változtatni fogunk pár dolgot az IAM-ben, úgyhogy érdemes a **Sign in using root user email** gombra kattintva root felhasználóként bejelentkezni.
2. A jobb felső sarokban lévő profil menüben a **Turn on multi-session support** gombra kattintva bekapcsolhatod azt a lehetőséget, hogy egyszerre több fiókba is bejelentkezhess. Jelentkezz be a legutóbb létrehozott **developer** IAM felhasználóval is! A továbbiakban egyszerűen válthatsz a két felhasználó közt.



3. Lépj be az EC2-be, és indítsd újra az adminisztrációra használt virtuális gépet az **Instance state** » **Start instance** opciót választva! Ha elindult, kattints az Instance ID-jára, hogy lásd a részletes adatait!
4. Másold a publikus IP címet az SSH kliensedbe, és ne felejtsd el megadni az előző gyakorlaton létrehozott privát kulcsot sem! A későbbiekben szükségünk lesz rá, hogy elérjük a virtuális gép 8089-es portját, úgyhogy a csatlakozás előtt állíts be rá SSH port forwardingot! Ezt PuTTY-ban a **Connection** » **SSH** » **Tunnels** menüben teheted meg:
 - 4.1 a Source port tetszőleges lehet, ezt írod majd be a saját gépeden
 - 4.2 a Destination a virtuális gép *privát* IP címéből és a 8089-es portból áll



- 4.3 kattints az **Add** majd a **Open** gombra, és lépj be az **ec2-user** felhasználóval!
5. A következő parancsokat használtuk legutóbb is a klaszter létrehozására EKS-ben. Amíg

ez fut, addig megyünk tovább az elmélettel, de szólj, ha hibát dob!

```
eksctl create cluster --region=eu-north-1 --name=rocket-chat \
--enable-auto-mode
# a kubeconfig fájlt aktualizálása
aws eks update-kubeconfig --region eu-north-1 --name rocket-chat
```

1.2. Deklaratív konfiguráció és YAML

Ha ugyanazt a konfigurációt szeretnénk több gépre alkalmazni, akkor kézenfekvőnek tűnik írni egy Bash, Python vagy PowerShell szkriptet. Ekkor az *imperatív* paradigmát követjük, vagyis megmondjuk, hogy milyen változtatásokat kell egymás után elvégezni a rendszeren. Ezzel szemben az 1. gyakorlaton használt *YAML*¹ egy *deklaratív* konfigurációs nyelv. Csak azt adjuk meg, hogy milyen végállapotba szeretnénk hozni a rendszert, az oda vezető lépéseket már nem nekünk kell kitalálnunk.

Alapvető egysége a kulcs-érték pár, például megadtuk a `rocket-chat.yaml`-ben, hogy a szolgáltatás milyen URL-en lesz elérhető:

```
host: k8s-(...).eu-north-1.elb.amazonaws.com
```

Az ugyanarra a komponensre vonatkozó részleteket behúzással adhatjuk meg, a belépési szabályzatot például így adtuk meg:

```
ingress:
  enabled: true
  ingressClassName: "alb"
```

Ha egy kulcshoz nem csak egy érték tartozhat, akkor gondolatjelekkel jelölhetjük az egyes elemeket, mint például az adatbázis jelszavait:

```
mongodb:
  enabled: true
  auth:
    passwords:
      - rocketchat
      - rocketchat-two
```

Egy szoftver konfigurációs tere rengeteg kulcs-érték párból állhat. Nekünk nem kell mindet megadnunk, csak amit meg szeretnénk változtatni. A `kubectl apply` parancs és a Helm összefésüli a saját értékeinket az alapértelmezettekkel.

1.3. Feladat: Rocket.Chat telepítése (újra)

1. Töltsd le és csomagold ki az első két gyakorlat segédfájljait!

```
curl http://vm.fured.cloud.bme.hu:16975/gyak2.tar.gz --output gyak2.tar.gz
tar -xf gyak2.tar.gz
```

2. A következő parancsokkal konfiguráld a perzisztens tárolást és a belépésvezérlőt!

¹A rövidítés eredetileg a *Yet Another Markup Language*-et jelentette, egy népszerű előtagot használva. Mivel nem jelölőnyelv, ez utólag *YAML Ain't Markup Language*-re változott. Az ehhez hasonló rekurzív rövidítéseknek szintén [réggi hagyománya](#) van a szakmában.

```
kubectl apply -f storage-class.yaml
kubectl apply -f ingress-class-params.yaml
kubectl apply -f ingress-class.yaml
kubectl apply -f ingress.yaml
```

3. A `kubectl get ingress` parancssal kérd le a belépésvezérlő URL-jét!

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
rocket-chat-ingress	alb	*	(URL)	80	8s

Állítsd be ezt a `host` értékeként a `rocket-chat.yaml` konfigurációs fájlban!

4. A következő parancsokkal telepítsd a Rocket.Chatet!

```
helm repo add rocketchat https://rocketchat.github.io/helm-charts
helm install rocketchat -f rocket-chat.yaml rocketchat/rocketchat
```

5. A Helm csak elindítja a folyamatot, időbe telik mire a konténerek ténylegesen működésbe jönnek. Emiatt még semmi nem érhető el az előbb használt URL-en, de jegyezd meg, mert onnan fogunk tovább indulni!

2. Terheléspróba

A mérnöki munka során sokfajta követelménynek meg kell felelnünk, például egyensúlyt kell találnunk a megbízható működés és a költséghatékonyság közt. Úgy kell megtervezni például egy hidat, hogy a csúcsgalimat is kibírja, de ne kerüljön a kelleténél több adóforintba. Hasonló a helyzet informatikai rendszereknél is, költséghatékonysan kell megfelelő minőségű szolgáltatást nyújtatnunk.

A pénzügyi oldal elég triviálisan számszerűsíthető, azt pedig *terheléspróbával* (*load test*) tudjuk igazolni, hogy a rendszer nem omlik össze a várt forgalomtól. Erre nem csak az átadáskor lehet szükség, hanem amikor megváltoznak a követelmények. A hidas példánál maradván ilyen volt, amikor a [Szabadság híd terheléspróbája](#) olyan villamosokkal, amik eddig ott nem közlekedtek. Mi a gyakorlaton pedig terheléspróbával fogjuk igazolni, hogy a beállított horizontális skálázódás megvalósul.

2.1. Rocket.Chat REST API

A Rocket.Chat az 1. gyakorlaton látott webes felületen kívül az összes jelentős asztali és mobil platformra elérhető kliensen keresztül is használható. Ezek egy [REST API](#)-n keresztül kommunikálnak a szerverrel, amin keresztül az összes jelentős funkció elérhető. További részletekért olvasd el a dokumentációt!

2.2. Locust

A [Locust](#) keretrendszer segítségével nagy számú felhasználót tudunk szimulálni, ezzel terheléspróbanak alávetve a rendszert. Ehhez egy `locustfile.py` fájlban le kell írunk egyetlen felhasználó tipikus viselkedését. A kiterjesztésből valószínűleg már kitaláltad, hogy mindezt Python nyelven tesszük. A terheléspróba során ebből fog rengeteg példányt párhuzamosan futtatni a keretrendszer.

Az [1.3.](#) szakaszban letöltött `gyak2.tar.gz` két Python fájlt is tartalmaz. A `test_setup.py` segítségével létre tudsz hozni 50 felhasználói fiókot a klasztereden, ezeket használjuk majd a terheléspróba. Ezen kívül van egy előre megírt `locustfile.py`, amely egy olyan felhasználót szimulál, aki:

1. A terheléspróba elején bejelentkezik.
2. Ezután 1–5 másodpercenként
 - vagy lekéri az összes aktív felhasználót,
 - vagy véletlenszerűen választ egyet a másik 49 felhasználóból, és neki küld 10 privát üzenetet.
3. A terheléspróba végén kijelentkezik.

Nyisd meg a példakódot, és keresd meg hogy hol használja a keretrendszer következő fontosabb elemeit:

RocketChatUser Ezen az osztályon belül definiáljuk egyetlen felhasználó tipikus viselkedését.

HttpUser Egy olyan őssztály, amely megkönnyíti HTTP/REST API-k terheléspróbáját.

on_start, on_stop A terheléspróba elején illetve végén egyszer lefutó függvények. A példában itt jelentkezik be és jelentkezik ki a felhasználó.

@task-kal dekorált metódus Ezek adják meg a szimulált felhasználó által végzett egyes műveleteket. A példában ebből kettő van, a **list_active_users** lekéri az összes aktív felhasználót, míg a **send_dm** privát üzeneteket küld. Teljes körű terheléspróba-hoz ezeknél sokkal többre lenne szükség, viszont nincs statisztikánk valódi felhasználói viselkedésről, amire ezeket alapozni lehetne.

wait_time Megadja, hogy a rendszer milyen időközönként hívjon meg egyet a **@task**-kal dekorált metódusok közül. A dekorátor paraméterében megadhatjuk, hogy milyen eséllyel válassza az adott metódust, ez alapértelmezetten 1.

between Paraméterül kapja a legrövidebb és a leghosszabb várakozási időt, egy véletlenszerűen ezek közé eső pillanatban hívódik meg az egyik **@task**-kal dekorált metódus.

self.client REST kliens objektum, amelynek **get**, **post** stb. metódusaival az azonos típusú kérést küldhetjük a terhelt rendszernek. Külön paraméterekkel a kérés fejlécét és törzsét is megadhatjuk.

A Rocket.Chat REST API-jának van egy különlegessége: a felhasználónévvel és jelszóval történő [bejelentkezésre](#) egy azonosítót és egy tokent kapunk válaszul. Ezeket minden további hívás fejlécében el kell küldenünk, ezzel azonosítva a felhasználót. A **locustfile.py**-ban ennek használatára is látsz példát.

A Locust tesztetket fejleszthetjük a saját gépünkön, és debugolás céljára praktikus lehet onnan futtatni. Éles méréseket viszont már nem érdemes innen végezni, hiszen kiszámíthatatlan a hálózati kapcsolat minősége a gépünk és a mért alkalmazás közt. Nem tudnánk összehasonlítani a ma mért eredményeket a holnapiakkal. A gyakorlaton ezért egy olyan virtuális gépről fogunk mérni, ami ugyanabban az adatközpontban van, mint a Kubernetes klaszter. A T3 példány típusok legfeljebb² [5 Gb/s sávszélességet](#) használhatnak, a klaszter alatt futó példányok pedig még többet, így a mai egyszerű terheléspróbáink során a hálózati kapcsolat biztosan nem lesz szűk keresztmetszet.

Nagyobb rendszerek terheléspróbájakor egyetlen virtuális gépet hamar ki lehet nőni. A Locust ezért [támogatja](#) a párhuzamos (egy gépen több folyamat) és az elosztott (több gépen legalább egy-egy folyamat) működést. Konténerekből is futtathatjuk a terheléspróbát, akár egy külön Kubernetes klaszteren. Ha már ez a megoldás is szűkösnek bizonyul, arra az esetre nyújtja az Amazon a [Distributed Load Testing on AWS](#) szolgáltatását.

²Ez azt jelenti, hogy a virtuális gép csak rövid időszakokra, burst-ös adatforgalomra használhat ekkora sávszélességet. Léteznek olyan példány típusok, amelyek folyamatosan garantálnak egy adott sávszélességet, de azok értelemszerűen drágábbak.

2.3. Feladat

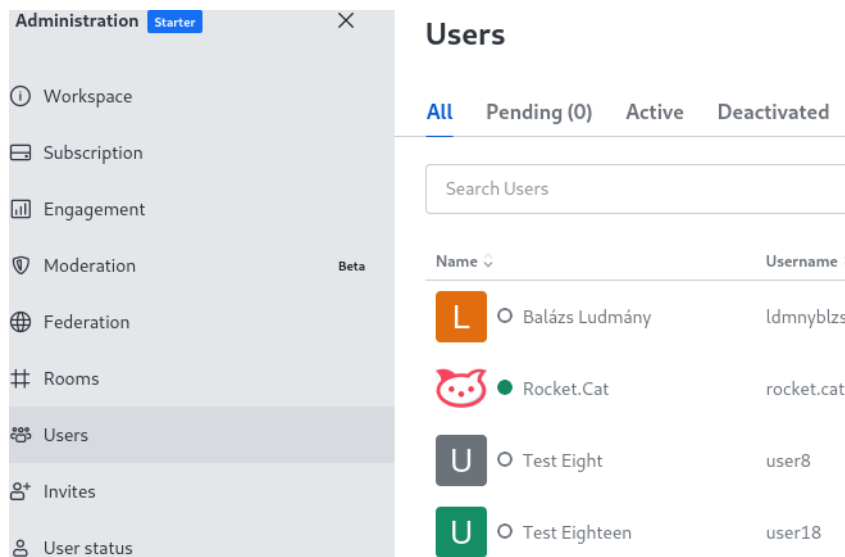
1. Ennyi idő alatt már el kellett indulnia a konténereknek, úgyhogy nyisd meg egy böngészőből a szolgáltatás URL-jét! Menj végig a Rocket.Chat telepítő varázslóján!
2. A Rocket.Chat alapértelmezetten korlátozza, hogy a REST API-n keresztül egységnyi idő alatt hány kérést indíthatunk, amit a terheléspróba idejére ki kell kapcsolnunk. A bal felső sarokban a 3 pöttyre kattintva nyisd meg az adminisztrátori menüt, majd kattints a **Workspace** gombra! Alul **Settings**, majd keress rá a Rate Limiterre, és nyisd meg! Az API Rate Limiter pontban kapcsold ki az Enable Rate Limiter opciót, majd kattints a **Save changes** gombra!
3. Ezután visszatérünk a virtuális gép SSH kapcsolatához. Először is telepítenünk kell a **pip** csomagkezelőt és néhány Python csomagot:

```
sudo yum install python3-pip
pip install locust num2words
```

4. A terheléspróbához létrehozunk 50 felhasználót. Ebben segít a gyakorlat fájljai közül a következő szkript:

```
python3 test_setup.py http://URL USER "PASSWORD"
```

Be kell helyettesíteni a szolgáltatás URL-jét, illetve a Rocket.Chat felhasználóneved és jelszavad. Utóbbit érdemes idézőjelek közé tenni, hogy ne zavarják meg az esetleges speciális karakterek a shellt. Az új felhasználókat a webes felületen is meg tudod nézni:



5. Add ki a **locust** parancsot a virtuális gépen, a saját gépeden pedig nyisd meg a **localhost:8089** URL-t! A Locust webes felülete fogad:

The screenshot shows the Locust web interface. At the top is a green header with the Locust logo and a hamburger menu icon. Below the header, the text "Start new load test" is displayed. There are three input fields: "Number of users (peak concurrency) *" with the value "50", "Ramp up (users started/second) *" with the value "1", and "Host" with the value "http://k8s-default-rocket-b774c18e8b-1679694394.eu-north-1.elb.amazonaws.com". Below these is an "Advanced options" section with a dropdown arrow, containing a "Run time (e.g. 20, 20s, 3m, 2h, 1h20m, 3h30m10s, etc.)" field with the value "2m". At the bottom is a large green button labeled "START".

6. A következő beállításokat kell megadni:

- összesen 50 felhasználót szimulálunk,
- másodpercenként 1 új felhasználó csatlakozik,
- a Host a szolgáltatásunk URL-je,
- 2 percig futtatjuk a terheléspróbát.

7. Kattints a `start` gombra, hogy elindítsd a terheléspróbát!

3. Állapotmentes mikroszolgáltatások automatikus skálázása

3.1. Alapfogalmak

Pod A platformszolgáltatáson futó mikroszolgáltatásainkat alkalmazáskonténerként futtatjuk. Ezzel a módszerrel a platform naprakészen tartásának felelőssége átkerül a platformszolgáltatótól a fejlesztőre. Ehhez létre kell hoznunk egy *konténer képet* (*container image*). Ez egy olyan csak olvasható bináris fájl, amely tartalmazza az összes szoftverkomponenst és a futtatásukhoz szükséges metaadatokat.

Ez nagyon hasonlít arra az infrastruktúra-szolgáltatásoknál, ahogy egy lemezképből létrehozunk egy virtuális gépet. A cloud-init segítségével kontextualizálni kellett a gépet, például megadni, hogy melyik virtuális hálózatra csatlakozzon, DHCP-t használjon-e vagy statikus IP címet stb. Az alkalmazáskonténerek esetében is felmerülnek hasonló kérdések, de ezekkel eddig nem foglalkoztunk, mert a Helm elvégzett mindent helyettünk.

A Kubernetes legkisebb futtatható egységei a *podok*, amik konténereket tartalmaznak, illetve az előbb említett metaadatokat is, amik a futtatásukhoz szükségesek. Mivel az esetek túlnyomó többségében podonként egyetlen konténert futtatunk, a két fogalom sokszor felcserélhető egymással.

Erőforrás kérés (resource request) és erőforrás határ (resource limit) Minden pod-hoz hozzárendelhetjük, hogy bizonyos erőforrásokból (pl. CPU, RAM) várhatóan mennyit fog fogyasztani, ez az erőforrás kérés (resource request), illetve hogy mennyit fogyaszthat legfeljebb, ez az erőforrás határ (resource limit).

A RAM felhasználást nem meglepő módon bájtban, kilobájtban, megabájtban stb. adhatjuk meg. A CPU felhasználást magonként adjuk meg, vagyis ennél az 1-es érték azt jelenti, hogy egy mag folyamatosan az adott podot futtatja. Elsőre szokatlan lehet, de itt is használhatunk SI prefixumokat, vagyis például 200 milliCPU annyit jelent, hogy az egyik mag az idejének 20%-át tölti az adott pod futtatásával.

Replikahalmaz (replica set) A horizontális skálázódás legalsó absztrakciós szintje. Azt felügyeli, hogy egyfajta *állapotmentes* podból mindig egy meghatározott számú fusson. Ha növeljük a podok számát vagy valamilyen hiba következtében az egyik kiesik, akkor egy *pod sablonból* (*template*) indít újakat. Fontos, hogy a felügyelt podoknak teljesen azonosnak kell lenniük, vagyis *állapottal rendelkező* (pl. adatbázis) podokat nem tehetünk replikahalmazba. Arról a következő gyakorlaton lesz szó, hogy ezeket az eseteket hogy kell kezelni.

Telepítés (deployment) A *telepítés* (*deployment*) egy replikahalmazt felügyel, és olyan plusz szolgáltatásokat valósít meg, mint például az automatikus horizontális skálázás.

Vezérlő (controller) A modell–nézet–vezérlő vagy angolul model–view–controller (MVC) tervezési mintával már találkoztál korábbi tanulmányaid során. A vezérlő feladata az események kezelése, például amikor a felhasználó rákattint a **B** gombra Wordben. Ezzel azt kommunikálja az alkalmazás felé, hogy szeretné félkövérré tenni a kijelölt szöveget. Ilyenkor a vezérlő feladata a kívánt állapotra hozni a dokumentumot leíró modellt, amit végül a nézet jelenít meg.

Hasonló elven működnek a *Kubernetes vezérlők* (*controller*) is, amik modell és nézet helyett a klasztert irányítják. Egérkattintás és hasonló események helyett itt deklaratív konfigurációs fájlok segítségével írjuk le a rendszer *elvárt állapotát* (*desired state*), a vezérlő pedig úgy avatkozik be a rendszerbe, hogy az *aktuális állapot* (*current state*) ehhez egyre közelebb kerüljön.

Az 1. gyakorlat 6.2. szakaszában már használtuk a belépésvezérlőt (ingress controller). Az elvárt állapot esetünkben egyszerű volt, mivel minden forgalmat a szolgáltatásunk 80-as portjára irányítottunk.

Horizontal Pod Autoscaler (HPA) A hosszú de beszédes nevű *Horizontal Pod Autoscaler* egy olyan vezérlő, amely figyeli az általa felügyelt telepítésekben a podok átlagos erőforrás felhasználását, és addig növeli vagy csökkenti a replikahalmaz méretét, hogy ezek egy megadott célértéket a lehető legjobban megközelítsék. A célértéket általában a replikahalmazt alkotó podok erőforrás határának százalékában adjuk meg. Az algoritmus részleteire előadáson még visszatérünk.

3.2. A korlátlan automatikus skálázódás veszélyei

A manuális skálázás egyik hátránya, hogy egy rosszindulatú aktor *szolgáltatásmegtagadással járó* (*denial of service* vagy *DoS*) vagy közismertebb nevén *túlterheléses* támadást indíthat a klaszterünk ellen. Ennek több fajtája létezik, de a cél mindig az, hogy a rendszer valamilyen szűk keresztmetszetét (pl. sávszélesség, számítási kapacitás) telítve legitim felhasználók számára elérhetatlenné tegyék a szolgáltatást. Automatikus skálázás esetén sokkal nehezebb ilyen szűk keresztmetszetet találni, hiszen újabb erőforrások bevonásával nagy mennyiségű rosszindulatú kérést is le tud kezelni a rendszer. Ezeket az erőforrásokat viszont ki kell fizetnünk, vagyis a bankszámlánk egyenlege válik szűk keresztmetszetté, amit *gazdasági fenntarthatóság megtagadásával járó* (*economic denial of sustainability* vagy *EDoS*) támadásnak hívunk.

3.3. Feladat: manuális skálázás

A Rocket.Chat lehetővé teszi, hogy konstans értékre rögzítsük a replikahalmaz méretét. Először ezt fogjuk kipróbálni.

1. Szerkeszd a `rocket-chat.yaml`-t:
 - 1.1 Töröld vagy kommentezd ki a `replicaCount` sort!
 - 1.2 A `microservices` alatt add meg, hogy az `authorization` szolgáltatás 2 példányban fusson:

```
microservices:
  enabled: true
  presence:
    authorization: 2
```

2. Alkalmazd a módosított konfigurációt:

```
helm upgrade rocketchat -f rocket-chat.yaml rocketchat/rocketchat
```

3. A `kubectl get deployment` paranccsal meg tudod nézni, hogy az egyes mikroszolgáltatások hány példányban futnak.

3.4. Feladat: automatikus skálázás

1. A `rocket-chat.yaml`-ben most nem a replikák darabszámát, hanem az `authorization` mikroszolgáltatás kért és maximális CPU használatát. A szemléletesség kedvéért alacsony értékeket választunk:

```
microservices:
  enabled: true
  authorization:
    resources:
      requests:
        cpu: 100m
      limits:
        cpu: 200m
```

2. Alkalmazd a módosított konfigurációt:

```
helm upgrade rocketchat -f rocket-chat.yaml rocketchat/rocketchat
```

3. A következő paranccsal megadjuk a HPA-nak, hogy

- az összes replikára vett átlagos CPU használat célértéke a kért CPU használat 50%-a,
- legalább 1, legfeljebb 3 podot szeretnénk futtatni.

```
kubectl autoscale deployment rocketchat-authorization --cpu=50% \
--min=1 --max=3
```

4. A `kubectl get hpa` paranccsal láthatod, hogy kezdetben 1 poddal indít a HPA.
5. A következő paranccsal a webes felület nélkül tudod 50 felhasználóval, 1 másodperces felfutási idővel 2 percig futtatni a Locust (az URL helyére értelemszerűen a saját Rocket.Chat telepítésedet helyettesítve):

```
locust --headless -u 50 -r 1 -t 2m -H http://URL
```

6. Add ki újra a `kubectl get hpa` parancsot, amivel most már több mint 1 podot kell kapnod.

3. gyakorlat

Infrastructure-as-Code

Eddig az Elastic Kubernetes Service Auto Mode-jának segítségével egy olyan klaszteren dolgoztunk, aminek felügyeletét nagyrészt az Amazon automatizált rendszerei végezték helyettünk. Ezzel szemben a mai gyakorlaton Terraform segítségével indítunk majd egy virtuális gépet, ami-re Ansible segítségével magunk telepítjük a K3s nevű Kubernetes disztribúciót, amire ismét Terraform segítségével telepítünk egy Nginx webszervert futtató podot.

1. Feladat

1. Indíts egy virtuális gépet az EC2-ben az 1. gyakorlat 7.4-es pontjában leírtak szerint. Egyetlen különbség, hogy a 3. gyakorlathoz tartozó `user-data.sh` szkriptet használd, ami telepíti a Terraformot és az Ansible-t is.
2. Jelentkezz be a gépre SSH-n keresztül!
3. A következő parancsok kiadásával győződj meg róla, hogy mindent sikerült-e telepíteni:

```
terraform -v
ansible --version
```

4. Töltsd le és csomagold ki a gyakorlat forrás fájljait:

```
curl http://vm.ik.bme.hu:3780/gyak3.tar.gz -o gyak3.tar.gz
tar -xf gyak3.tar.gz
```

5. Az `infrastructure` mappa tartalmazza a virtuális gépet létrehozó Terraform modult. Nézd át a forráskódját, majd a következő parancsokat az `infrastructure` mappából kiadva indítsd el a gépet:

```
terraform init
terraform plan
terraform apply
```

6. A következő parancsokkal megbizonyosodhatsz arról, hogy a virtuális gép elindult:

```
aws ec2 describe-instances --query 'Reservations[].Instances[].[InstanceId,PublicIpAddress]'
terraform state list
terraform state show aws_instance.cluster
```

7. A `kubernetes` mappa tartalmazza azokat az Ansible fájlakat, amelyek segítségével egy egy node-os Kubernetes klasztert tudunk létrehozni. A következő parancs segítségével meggyőződhetsz róla, hogy az Ansible control node eléri a felügyelt node-ot:

```
ansible all -i inventory.yml -m ping
```

8. A következő paranccsal telepítheted a K3s-t:

```
ansible-playbook -i inventory.yml k3s-install.yml
```

9. Ezzel pedig lemásolhatod a telepítő által generált kubeconfig-ot:

```
ansible-playbook -i inventory.yml fetch-kubeconfig.yml
```

10. Végül telepítsd az Nginx-et a `nginx` mappa tartalmát használva:

```
terraform init  
terraform plan  
terraform apply
```